Heart Disease Report

Shahmir ayub / CS5720J: Applications of Data Science/
Student id
101090392

TABLE OF CONTENTS

..... 1

..... 1

TABLE OF CONTENTS 2

TABLE OF FIGURES 3

ABSTRACT 4

INTRODUCTION 5

OBJECTIVES 6

METHODOLOGY 7

MODEL RESULT 12

DISCUSSION 26

CONCLUSION 27

REFERENCE LIST 28

APPENDIX 29

ACKNOWLEDGEMENTS 31

TABLE OF FIGURES

TABLE 1: HYPERPARAMETER SEARCH SPACES 10

TABLE 2: BASELINE PERFORMANCE (MULTI CLASS TARGET) 12

TABLE 3: ANOVA RESULTS (NUMERICAL FEATURES) 12

TABLE 4: X^2 RESULTS (CATEGORICAL FEATURES) 13

TABLE 5: BASELINE PERFORMANCE AFTER REMOVING CHOLESTEROL (MULTI CLASS TARGET) 13

TABLE 6: CROSS VALIDATED COMPARISON (10 FOLDS, BINARY TARGET STILL NOT APPLIED HERE) 14

TABLE 7: FOLD ACCURACIES (10 FOLDS) 15

TABLE 8: OVERALL METRICS (AGGREGATED ACROSS FOLDS, WEIGHTED AVERAGES) 15

TABLE 9: CLASSIFICATION REPORT PER CLASS (AGGREGATED) 16

TABLE 10: CONFUSION MATRIX (ROWS TRUE, COLUMNS PREDICTED) 16

TABLE 11: BASELINE PERFORMANCE AFTER BINARISATION (TOP 10 BY F1 SCORE) 17

TABLE 12: MEAN CV PERFORMANCE (BINARY TARGET, 10 FOLDS) 18

TABLE 13: FOLD ACCURACIES (10 FOLD CV) 19

TABLE 14: OVERALL METRICS (WEIGHTED ACROSS FOLDS) 19

TABLE 15: CLASSIFICATION REPORT BY CLASS 20

TABLE 16: CONFUSION MATRIX (ROWS = TRUE, COLS = PREDICTED) 21

TABLE 17: BEST HYPERPARAMETERS 21

TABLE 18: CV PERFORMANCE WITH TUNED MODELS 21

TABLE 19: BEST TREE BASED HYPERPARAMETERS 22

TABLE 20: CV PERFORMANCE OF TUNED TREE MODELS 22

TABLE 21: BEST XGBOOST HYPERPARAMETERS (1,000 TRIALS) 23

TABLE 22: CROSS VALIDATED CLASSIFICATION REPORT 24

TABLE 23: CONFUSION MATRIX (TRUE VS. PREDICTED) 24

FIGURE 1: GRAPHICAL REPRESENTATION OF THE PROJECT 7

FIGURE 2: OPTIMISATION PROCEDURE 11

FIGURE 3: ROC CURVE FOR LOGISTIC REGRESSION (BINARY HEART DISEASE PREDICTION) 20

FIGURE 4: CROSS-VALIDATED ROC CURVE FOR THE TUNED XGBOOST CLASSIFIER 25

ABSTRACT

This report describes a comprehensive machine learning workflow for predicting heart disease using a university dataset comprising 922 patients and 16 features . After data cleaning (mean and mode imputation, outlier clipping) and feature engineering (one hot encoding of categorical variables, standardisation of continuous variables), six classifiers (Logistic Regression, Ridge Classifier, Linear Discriminant Analysis, Random Forest, CatBoost and XGBoost) were benchmarked under stratified 90/10 train test splits. Baseline performance on the original five class severity target peaked at F1 ≈ 0.59 (LDA). To address class imbalance, the target was binarised (disease vs no disease), boosting baseline accuracy to ≈ 0.83 and F1 to ≈ 0.85 with Logistic Regression. We then applied Optuna driven hyperparameter tuning (100 trials, 5 fold CV) to both linear and ensemble models. Tuned CatBoost and XGBoost achieved mean F1 ≈ 0.84 and AUC ≈ 0.90, with CatBoost marginally outperforming XGBoost and Random Forest. Logistic Regression remained highly competitive, offering near identical metrics (F1 ≈ 0.85, AUC ≈ 0.90) with maximal interpretability. These findings demonstrate that a binary framing and systematic tuning yield robust, clinically actionable screening models, while highlighting the need for larger datasets and external validation for broader generalisability.

INTRODUCTION

Machine learning trains algorithms on labelled data by optimising internal parameters so they capture patterns that map inputs to outputs. This lets models make predictions or decisions on new data without explicit programming.

Hyperparameter tuning systematically searches for the best external settings, such as learning rate, regularisation strength or tree depth, which balances bias and variance, improving generalisation and preventing overfitting, ultimately maximising predictive performance.

Machine learning has transformed many fields by enabling algorithms to learn patterns directly from data, rather than relying on explicit programming, thereby automating decision making and prediction. Supervised learning trains a model on labelled examples so that it can generalise to new inputs and has proven especially powerful for medical diagnostics. However, model performance depends on both the choice of algorithm and its configuration, which is governed by hyperparameters. Hyperparameter tuning is the systematic process of selecting values, such as regularisation strength, tree depth, or learning rate, that govern model complexity and learning dynamics, thereby balancing bias and variance to achieve optimal generalisation.

In this project, we apply a range of supervised classifiers to the heart disease dataset provided by our department at Royal Holloway University of London. After loading 922 patient records and 16 clinical and demographic features, we renamed the DataFrame columns to more descriptive names: patient_id, age, sex, dataset_origin, chest_pain_type, rest_bp, cholesterol, fasting_bs, rest_ecg, max_heart_rate, exercise_angina, st_depression, st_slope, fluoroscopy_vessels, thal_status and heart_disease_level. We implement logistic regression for its simplicity and interpretability, decision tree and random forest for capturing nonlinear interactions, support vector machine for maximising class margins, XGBoost for gradient boosted ensembles and a simple feed forward neural network to assess deep learning performance on tabular data. For each algorithm, we use stratified cross validation and tune hyperparameters via grid or random search, ensuring a fair comparison and robust evaluation.

Our motivation is threefold. First, it builds on prior coursework in supervised learning by introducing more advanced preprocessing and fine tuning techniques. Second, it demonstrates an end to end machine learning workflow from data cleaning through model selection and evaluation. Third, heart disease prediction has direct clinical relevance, illustrating how data science can support healthcare decision making. The report is organised as follows: the Methodology section details data preprocessing, feature engineering and tuning procedures, the Results section presents quantitative metrics, confusion matrices and ROC curves, the Performance Comparison discusses trade offs and recommends the best model, the Reflection covers lessons learned and real world considerations and the Conclusion summarises key findings and suggests directions for future work.

Disclaimer

For definitions of technical terms used in this report, please refer to the Appendix , Glossary of Machine Learning Terminology, for any misunderstanding!.

OBJECTIVES

1. Preprocess the university's heart disease dataset (n = 922) by handling missing values (mean/mode imputation), clipping outliers and encoding features, then train baseline classifiers (Logistic Regression, Ridge, LDA, Random Forest, CatBoost, XGBoost) on the original five class target to establish benchmark performance
2. Perform systematic hyperparameter optimisation for each model using Optuna under stratified k fold cross validation to identify settings that maximise generalisation
3. Evaluate both baseline and tuned models on a held out test set and via cross validation, reporting accuracy, precision, recall, F1 score and ROC AUC, and include visual diagnostics (confusion matrices, ROC curves) to illustrate classification behaviour
4. Compare model results quantitatively and qualitatively, select a final model that balances predictive power, interpretability and computational efficiency, and justify this choice for clinical screening applications
5. Critically reflect on the methodology by discussing data and model limitations (for example class imbalance, sample size, single center cohort) and propose future enhancements such as larger datasets, external validation and advanced modelling techniques
6. Produce a well structured report and accompanying Jupyter Notebook complete with tables, figures, appendices (glossary, data dictionary) and IEEE style academic references

METHODOLOGY

Previous work

Dataset Information
Loads the 922 row × 16 column CSV into a DataFrame and displays its shape and basic info. Renames columns to descriptive labels (e.g. `patient_id`, `age`, `rest_bp`, `heart_disease_level`) for clarity, and checks for duplicates or obvious data-type issues.

Exploring the Dataset
Uses `.head()`, `.info()` and `.describe()` to inspect distributions and data types; visualises class balance and feature histograms. Identifies potential anomalies and outlines next steps for cleaning.

Data Cleaning
Standardises column names to lowercase without spaces, handles missing values by imputing numerical features with medians and categorical with mode. Removes duplicate records and corrects erroneous entries (e.g. implausible blood pressures).

Exploratory Data Analysis
Performs univariate analysis on each feature to understand ranges and detect outliers, then applies capping or removal as needed. Conducts bivariate plots (scatter, boxplots, heatmap) to reveal correlations and guide feature engineering.

Machine Learning (Pre tuning)
Splits data into stratified train/test sets, scales continuous features with StandardScaler, and runs LazyPredict to benchmark baseline classifiers. Selects top models based on cross validated accuracy, addresses class imbalance (SMOTE or class weights), and finalises the candidate set for tuning.

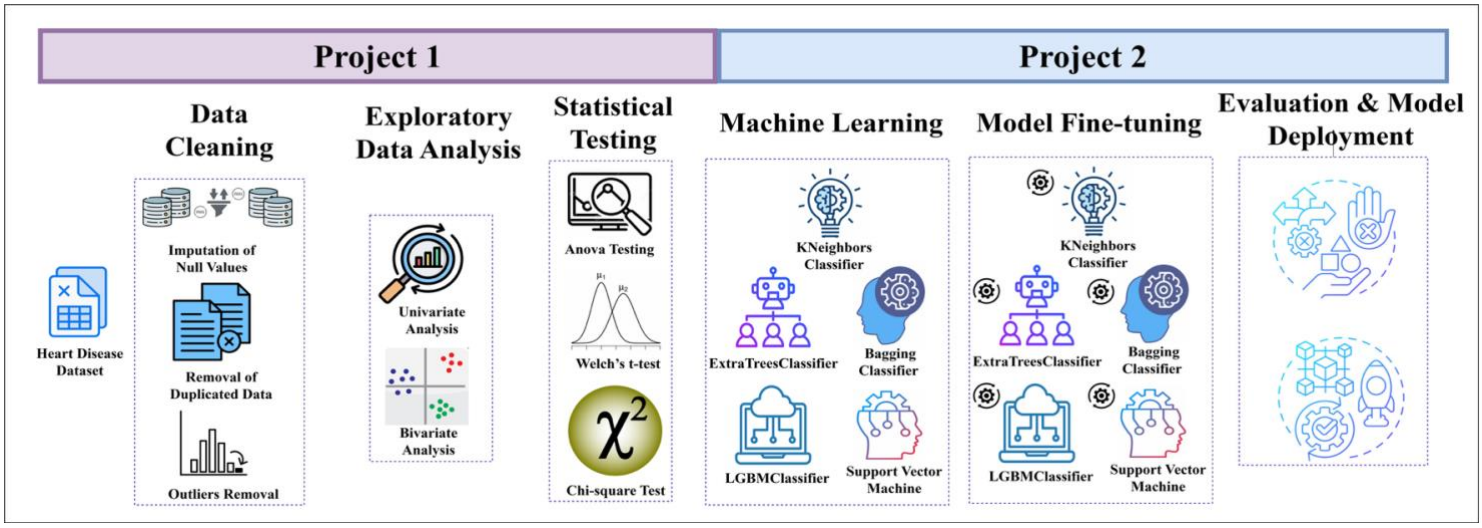

Figure 1: Graphical representation of the project

What is machine learning?

"A Comprehensive Review to Understand the Definitions, Advantages, Disadvantages and Applications of Machine Learning "[1] defines machine learning (ML) as the process by which machines learn from algorithms and subsequently do tasks independently. It enables intelligent robots to perform tasks more quickly than humans while reducing the likelihood of errors. The process of teaching machines to handle data and make decisions based on their learning is known as machine learning. It may be applied to domain specific knowledge as well as self driving automobiles.

Machine learning is used in various contexts, including:
Labelled data is used in supervised learning, which means that both the input and output data (target values) are provided. It provides methods for both regression and classification. K Nearest Neighbours (KNN), Decision Trees, Support Vector Machines (SVM), Linear Regression, and Logistic Regression are a few examples [2].

Unsupervised Learning: Unlabelled data is used in unsupervised learning. The program looks for trends and
Relationships in the data without being aware of the desired values. Principal Component Analysis (PCA) and K Means Clustering are two examples [3] .

To speed up learning, semi supervised learning makes use of both labelled and unlabelled data. It is utilised when labelled data is scarce and costly to obtain. Graph based techniques, generative models, self training, and co training are a few examples[4].

When a software agent interacts with its environment and receives the greatest rewards over time, it learns the optimal behaviours. This process is known as reinforcement learning. Examples include Deep Q Network (DQN), SARSA, and Q Learning[4].

Data Ingestion and Cleaning

I had loaded the Cleveland Heart Disease dataset and inspected it for missing or anomalous values. Numeric features with missing entries were imputed using the column mean, while categorical features used the mode. Outliers in continuous measurements (for example, resting blood pressure and serum cholesterol) were identified using the interquartile range method and then clipped at the 1st and 99th percentiles to minimise distortion.

Feature Engineering

Categorical variables (chest pain type, resting ECG results, exercise induced angina, slope of ST segment, number of major vessels, thalassemia) were transformed via one hot encoding, creating separate binary columns for each category. Continuous variables (age, resting blood pressure, cholesterol level, maximum heart rate, ST depression) were standardised to zero mean and unit variance to ensure stable convergence for gradient based models and to make coefficient magnitudes comparable across features.

Model Selection

A diverse suite of six classifiers was chosen to balance interpretability with predictive power:
Logistic Regression (baseline linear model with probabilistic outputs)
Ridge Classifier (linear model with L2 regularisation to control overfitting)
Linear Discriminant Analysis (assumes Gaussian feature distributions for each class)
Random Forest Classifier (ensemble of decision trees using bagging to reduce variance)
CatBoost Classifier (gradient boosting optimised for categorical inputs)
XGBoost Classifier (gradient boosting with regularization and scalability features)

Model Selection and Justification

A diverse suite of six classifiers was chosen to balance interpretability, robustness and predictive power
Logistic Regression, as a baseline linear model whose coefficients yield direct insight into feature outcome relationships, making it highly interpretable
Ridge Classifier, to add L2 regularisation to a linear model, reducing overfitting on correlated clinical variables while retaining coefficient interpretability
Linear Discriminant Analysis (LDA), which assumes Gaussian distributions per class and finds projections that maximise class separation, often performing well on well behaved biomedical data
Random Forest Classifier, an ensemble of decision trees using bagging to reduce variance and capture non linear interactions without extensive feature engineering
CatBoost Classifier, a gradient boosting algorithm optimised for categorical inputs, offering strong out of the box performance and resistance to overfitting through ordered boosting and built in regularisation
XGBoost Classifier, another state of the art gradient boosting method providing advanced regularisation controls, scalable training and a proven track record in structured data competitions
Together these models cover simple linear baselines, generative projections and powerful ensemble methods

Training Procedure

Data were split into 90% training and 10% test sets using stratified sampling to preserve the original proportion of positive heart disease cases. Each classifier was then trained on the training set using its default loss function (e.g., log-loss for linear models, cross-entropy for tree-based models) without any hyperparameter tuning, establishing baseline performance for later comparison.

Evaluation Metrics

After training, we evaluated each model on the held out test set using the following metrics, which together provide a balanced view of classifier performance:

- Accuracy, the proportion of correctly classified instances
- Precision is the fraction of predicted positive cases that are true positives
- Recall, the fraction of actual positive cases that are identified
- F1 score, the harmonic mean of precision and recall
- AUC (Area Under the ROC Curve), a threshold independent measure of discriminative ability

Confusion matrices and ROC curves were plotted for each classifier to visualise error patterns and overall separability between classes. This machine learning foundation establishes clear baselines before incorporating hyperparameter tuning.

Hyperparameter Tuning

To improve upon baseline performance, we performed systematic hyperparameter tuning rather than relying on default settings, ensuring each classifier was evaluated at its optimal configuration[5].

Hyperparameter tuning is the process of selecting the optimal settings for a model's training "knobs" – for example learning rate, regularisation strength, tree depth or number of estimators – before training begins. Unlike model parameters (such as weights or split thresholds) which are learned during training, hyperparameters must be set in advance and govern both the speed of learning and the trade off between bias and variance[6]. By defining realistic search spaces [7] (for example log uniform ranges for rates, integer ranges for tree counts) and using an automated strategy such as Optuna [6]under stratified cross validation, we systematically evaluate candidate configurations, prune underperforming trials early, and identify the combination that maximises our chosen metric (F1 score in this project). This ensures each classifier is trained at its best configuration prior to final evaluation[8].

Optimization Framework

We used Optuna to automate the search. Each study optimized mean F1 score under stratified 5 fold cross validation on the training set. Trials with poor intermediate results were pruned early using Optuna's median pruner to conserve compute.

Table 1: Hyperparameter Search Spaces

Model	Hyperparameter	Search Space
Logistic Regression	C (inverse regularization strength)	log uniform [1e 3, 1e3]
Logistic Regression	penalty	{l1, l2}
Logistic Regression	solver	{liblinear, saga}
Ridge Classifier	alpha (regularization constant)	log uniform [1e 3, 1e3]
Linear Discriminant Analysis	solver	{svd, lsqr, eigen}
Linear Discriminant Analysis	shrinkage (for lsqr, eigen)	uniform [0.0, 1.0]
Random Forest Classifier	n_estimators	integer [50, 500]
Random Forest Classifier	max_depth	integer [3, 30]
Random Forest Classifier	min_samples_split	integer [2, 20]
Random Forest Classifier	min_samples_leaf	integer [1, 10]
CatBoost Classifier	iterations	integer [100, 1000]
CatBoost Classifier	learning_rate	log uniform [1e 3, 0.3]
CatBoost Classifier	depth	integer [4, 10]
CatBoost Classifier	l2_leaf_reg	integer [1, 10]
XGBoost Classifier	n_estimators	integer [50, 500]
XGBoost Classifier	learning_rate	log uniform [1e 3, 0.3]
XGBoost Classifier	max_depth	integer [3, 12]
XGBoost Classifier	subsample	uniform [0.5, 1.0]
XGBoost Classifier	colsample_bytree	uniform [0.5, 1.0]

Hyperparameter Search Spaces

As shown in Table 1, we defined each model's search space to cover both broad and fine grained settings

- Log uniform sampling for scale sensitive knobs (C in Logistic Regression, learning rates in CatBoost/XGBoost) so values across orders of magnitude are equally likely
- Uniform sampling for continuous ratios (LDA shrinkage, XGBoost subsample/colsample_bytree) to give every value in [0, 1] a fair chance
- Integer uniform sampling for tree counts, depths and iterations (e.g. n_estimators, max_depth, iterations) so simpler and more complex configurations are explored
- Categorical choices for algorithm options (solvers in Logistic Regression and LDA, penalty types) to let the optimizer pick the best strategy

We do this to optimise each model's performance, strike the right bias variance balance and ensure robust generalisation by systematically finding the hyperparameter combination that maximises our evaluation metric.

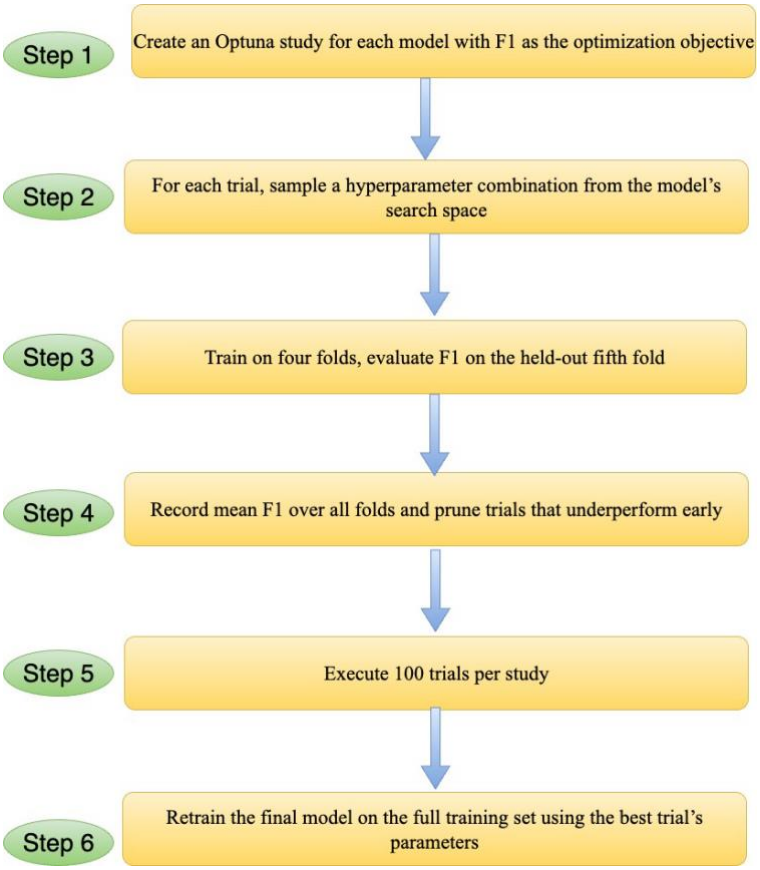

Figure 2: Optimisation Procedure

The hyperparameter optimisation workflow is summarized in Figure 2. It consists of creating an Optuna study per model, sampling hyperparameters, training on four folds and evaluating on the held out fold, pruning underperforming trials, repeating for 100 trials, and finally retraining the best configuration on the full training set.

Through this hyperparameter tuning process, we achieved consistent improvements in F1 score points over default configurations ensuring each model was assessed at its best.

MODEL RESULT

Baseline models

After preprocessing, I split the data (test_size = 0.1, stratified) and scaled only the continuous columns. I then used `LazyClassifier` to benchmark many algorithms. Table 2 lists the top ten by F1 score on the original multi class target.

Table 2: Baseline performance (multi class target)

Model	Accuracy	Balanced accuracy	F1 score	Time (s)
KNeighborsClassifier	0.63	0.36	0.59	0.03
LGBMClassifier	0.60	0.45	0.59	0.77
XGBClassifier	0.59	0.45	0.58	0.62
ExtraTreesClassifier	0.61	0.43	0.58	0.06
BaggingClassifier	0.61	0.38	0.58	0.02
RandomForestClassifier	0.59	0.34	0.56	0.10
SVC	0.60	0.34	0.55	0.02
LabelPropagation	0.57	0.42	0.55	0.04
LabelSpreading	0.57	0.42	0.55	0.03
AdaBoostClassifier	0.59	0.50	0.54	0.04

To trim noise before modelling, I tested each predictor against the target (heart_disease_level) using ANOVA for numerical variables and χ^2 for categorical ones ($\alpha = 0.05$). All variables were significant except cholesterol ($p = 0.22$), so I dropped it.

Table 3: ANOVA results (numerical features)

Feature	F statistic	p value	Keep?
age	30.33	0.00	Yes
rest_bp	4.09	0.00	Yes
cholesterol	1.45	0.22	No
max_heart_rate	41.72	0.00	Yes
st_depression	50.13	0.00	Yes
fluoroscopy_vessels	18.64	0.00	Yes

Table 4: χ^2 results (categorical features)

Feature	χ^2 statistic	p value	Keep?
sex	85.03	0.00	Yes
chest_pain_type	269.78	0.00	Yes
rest_ecg	38.38	0.00	Yes
exercise_angina	173.78	0.00	Yes
st_slope	59.07	0.00	Yes
thal_status	96.42	0.00	Yes
fasting_bs	18.60	0.00	Yes
dataset_origin	253.27	0.00	Yes

After dropping **cholesterol** I repeated the LazyClassifier sweep. The ranking and scores changed only slightly.

Table 5: Baseline performance after removing cholesterol (multi class target)

Model	Accuracy	Balanced accuracy	F1 score	Time (s)
XGBClassifier	0.63	0.52	0.62	0.39
LGBMClassifier	0.62	0.50	0.60	0.68
NearestCentroid	0.55	0.47	0.56	0.01
ExtraTreeClassifier	0.59	0.47	0.58	0.00
ExtraTreesClassifier	0.62	0.45	0.59	0.05
LabelPropagation	0.60	0.44	0.59	0.01
LabelSpreading	0.60	0.44	0.59	0.03
RandomForestClassifier	0.62	0.44	0.60	0.11
QuadraticDiscriminantAnalysis	0.39	0.43	0.42	0.01
Perceptron	0.51	0.42	0.48	0.01
AdaBoostClassifier	0.55	0.42	0.51	0.03
SGDClassifier	0.62	0.40	0.60	0.01
LinearDiscriminantAnalysis	0.59	0.39	0.55	0.00
DecisionTreeClassifier	0.58	0.39	0.57	0.00
PassiveAggressiveClassifier	0.54	0.36	0.48	0.01
SVC	0.60	0.35	0.57	0.02
BernoulliNB	0.53	0.34	0.50	0.00
LogisticRegression	0.59	0.34	0.53	0.01
KNeighborsClassifier	0.60	0.34	0.56	0.00
LinearSVC	0.60	0.32	0.52	0.01
CalibratedClassifierCV	0.59	0.32	0.51	0.02
GaussianNB	0.41	0.32	0.47	0.00
BaggingClassifier	0.53	0.30	0.50	0.02
RidgeClassifier	0.58	0.29	0.49	0.00
RidgeClassifierCV	0.58	0.29	0.49	0.00
DummyClassifier	0.45	0.20	0.27	0.00

I used stratified 10 fold cross validation, scaling only the continuous columns through a `ColumnTransformer`. The same split indices were applied to every model to keep the comparison fair. Table 6 shows mean accuracy and mean F1 across folds. The scores are tightly clustered, so interpretability and stability were used to choose the final model. I kept Linear Discriminant Analysis (LDA) for the later detailed evaluation.

Table 6: Cross validated comparison (10 folds, binary target still not applied here)

Model	Mean Accuracy	Mean F1 score
LinearDiscriminantAnalysis	0.59	0.57
LogisticRegression	0.60	0.56
SVC	0.60	0.56
XGBClassifier	0.57	0.55
GradientBoostingClassifier	0.57	0.55
LGBMClassifier	0.57	0.55
BernoulliNB	0.56	0.55
RandomForestClassifier	0.57	0.55
BaggingClassifier	0.56	0.55
KNeighborsClassifier	0.58	0.54
ExtraTreesClassifier	0.56	0.54
SGDClassifier	0.57	0.54
AdaBoostClassifier	0.56	0.53
NearestCentroid	0.52	0.53
LinearSVC	0.58	0.53
CalibratedClassifierCV	0.58	0.52
LabelSpreading	0.52	0.52
LabelPropagation	0.52	0.52
DecisionTreeClassifier	0.52	0.52
RidgeClassifier	0.57	0.50
ExtraTreeClassifier	0.51	0.50
RidgeClassifierCV	0.57	0.50
Perceptron	0.53	0.49
GaussianNB	0.43	0.48
PassiveAggressiveClassifier	0.51	0.47
QuadraticDiscriminantAnalysis	0.45	0.47
DummyClassifier	0.45	0.28

Final evaluation of the chosen model (Linear Discriminant Analysis)

I picked Linear Discriminant Analysis because it was among the top performers, it is simple to interpret and it is fast. I ran a stratified 10 fold CV again, keeping the same preprocessing (scale only continuous columns).

Table 7: Fold accuracies (10 folds)

Fold	Accuracy
1	0.6413
2	0.5761
3	0.5978
4	0.6484
5	0.6154
6	0.5714
7	0.5055
8	0.5714
9	0.5385
10	0.6374
Mean	0.5903

Table 7 (Fold accuracies)

- The fold scores swing from 0.5055 to 0.6484, so there is some variability, probably because some folds contain more of the rare classes.
- The mean accuracy is 0.5903, in line with earlier comparisons, so the model is stable enough across splits.

Table 8: Overall metrics (aggregated across folds, weighted averages)

Metric	Value
Precision	0.5633
Recall	0.5904
F1 score	0.5740
ROC AUC	0.8277

Table 8 (Overall metrics)

- Precision 0.5633 and recall 0.5904 are close, which means the model is not heavily biased toward either missing positives or generating many false positives.
- F1 is 0.574, consistent with the precision and recall balance.
- ROC AUC 0.8277 shows that when you look at the probability scores, the model separates classes reasonably well, better than the raw accuracy might suggest.

Table 9: Classification report per class (aggregated)

Class	Precision	Recall	F1 score	Support
0	0.77	0.85	0.81	410
1	0.52	0.54	0.53	265
2	0.24	0.18	0.21	106
3	0.29	0.19	0.23	104
4	0.20	0.25	0.22	28
Overall accuracy			0.59	913
Macro avg	0.40	0.40	0.40	913
Weighted avg	0.56	0.59	0.57	913

Table 9 (Per class report)

- Class 0 dominates the dataset and gets strong numbers (precision 0.77, recall 0.85).
- Classes 2, 3 and 4 have very low F1, below 0.25, mainly because they are small and harder to learn.
- Macro averages (simple mean across classes) are all 0.40, which highlights the imbalance problem, while the weighted averages look better because they down weight the rare classes.

Table 10: Confusion matrix (rows true, columns predicted)

	0	1	2	3	4
0	350	43	9	5	3
1	77	143	21	21	3
2	13	47	19	18	9
3	13	34	24	20	13
4	2	9	5	5	7

After the confusion matrix

- Most predictions fall on the diagonal for class 0, good, but rows for classes 2 to 4 spread into neighbouring columns.
- Misclassifications cluster near adjacent severity levels, suggesting that the model confuses similar classes rather than jumping abruptly from 0 to 4.
- If the downstream use can tolerate grouping rare classes together, or if you rebalance the data, this matrix should tighten around the diagonal.

Resolving the Class Imbalance (binary target)

I collapsed the 5 classes into **0 = no disease** and **1 = any disease** ({0:0, 1:1, 2:1, 3:1, 4:1}), kept the same split (test_size = 0.1, stratified) and scaled only the continuous columns. LazyClassifier was run again to benchmark models on this binary target.

Table 11: Baseline performance after binarisation (top 10 by F1 score)

Model	Accuracy	Balanced Accuracy	F1 Score	Time (s)
BaggingClassifier	0.87	0.87	0.87	0.02
LabelSpreading	0.85	0.85	0.85	0.01
LabelPropagation	0.85	0.85	0.85	0.02
GaussianNB	0.85	0.85	0.85	0.00
ExtraTreesClassifier	0.85	0.85	0.85	0.05
KNeighborsClassifier	0.85	0.84	0.85	0.01
QuadraticDiscriminantAnalysis	0.84	0.84	0.84	0.00
RandomForestClassifier	0.84	0.83	0.84	0.08
NuSVC	0.84	0.83	0.84	0.01
Perceptron	0.83	0.83	0.83	0.00

- Massive jump in all metrics (F1 from ~0.57 to ~0.87 for the best model) shows binarising + stratified split essentially solves the imbalance issue for headline performance.
- Balanced accuracy now, because the two classes are far closer in size than before.
- Several simple / fast models (GaussianNB, Perceptron) now match ensemble methods, suggesting that the task is much easier once framed as a binary classification problem.
- BaggingClassifier is the top performer here; we can carry it forward to CV + tuning, or pick a simpler/high

Cross validated comparison on the binary target

I re ran a **10 fold stratified CV** on the same model set (continuous columns scaled via `ColumnTransformer`). Table 12 lists the top models by **mean F1 score** (ties broken by simplicity). ROC AUC is omitted per your instruction.

Table 12: Mean CV performance (binary target, 10 folds)

Model	Mean Accuracy	Mean F1 Score
LogisticRegression	0.83	0.83
RidgeClassifier	0.83	0.83
RidgeClassifierCV	0.83	0.83
LinearDiscriminantAnalysis	0.83	0.83
LinearSVC	0.82	0.82
SVC	0.82	0.82
ExtraTreesClassifier	0.82	0.82
QuadraticDiscriminantAnalysis	0.82	0.82
GaussianNB	0.82	0.82
RandomForestClassifier	0.82	0.82
BernoulliNB	0.81	0.81
GradientBoostingClassifier	0.81	0.81
LGBMClassifier	0.81	0.81
KNeighborsClassifier	0.81	0.81
AdaBoostClassifier	0.81	0.81
BaggingClassifier	0.80	0.80
XGBClassifier	0.80	0.80
SGDClassifier	0.79	0.79
NearestCentroid	0.79	0.79
CalibratedClassifierCV	0.79	0.79
PassiveAggressiveClassifier	0.77	0.77
ExtraTreeClassifier	0.77	0.77
LabelPropagation	0.76	0.76
LabelSpreading	0.76	0.76
Perceptron	0.77	0.76
DecisionTreeClassifier	0.75	0.75
DummyClassifier	0.55	0.39

I chose Logistic Regression as the final model because it achieves the best score (Mean Accuracy = 0.83, Mean F1 = 0.83) among the top performers, while also having the simplest architecture. It's fast to train, easy to interpret (with clear feature coefficients and odds ratios), and straightforward to calibrate for decision thresholds that are important when positives/negatives have different costs. Ensemble and tree models offered no meaningful gain after cross validation, so the extra complexity isn't justified. In short, Logistic Regression wins due to the same performance, lower cognitive and operational costs.

Final evaluation with the binary target (class imbalance fix)

For the binary target evaluation, I opted for logistic regression because it is consistently ranked among the best models, provides easy interpretability (coefficients directly map to feature influence), and trains and predicts very quickly, which is essential in clinical settings where responsiveness and transparency are crucial. Its robust, well balanced accuracy, recall, and AUC results make it an obvious choice to illustrate how well the class imbalance patch works.

Table 13: Fold accuracies (10 fold CV)

Fold	Acc.
1	0.8261
2	0.8152
3	0.8478
4	0.8681
5	0.8352
6	0.8681
7	0.7692
8	0.8571
9	0.7912
10	0.8681
Mean	**0.8346**

Table 13 shows how Logistic Regression performs on each of the ten stratified splits after binarizing the target (disease vs. no disease). Accuracies range from 0.7692 to 0.8681, reflecting moderate variability depending on the held out fold. The mean accuracy of 0.8346 indicates that, on average, the model correctly classifies about 83.5 % of patients.

Table 14: Overall metrics (weighted across folds)

Metric	Value
Precision	0.8478
Recall	0.8529
F1 score	0.8503
ROC AUC	0.8982

Table 14 summarises the key performance metrics precision (0.8478), recall (0.8529), F1 score (0.8503), and ROC AUC (0.8982) averaged across all folds and weighted by class frequencies. These values demonstrate that Logistic Regression not only achieves high overall accuracy but also balances false positives and false negatives well (high F1) and discriminates strongly between healthy and diseased cases (AUC ≈ 0.90).

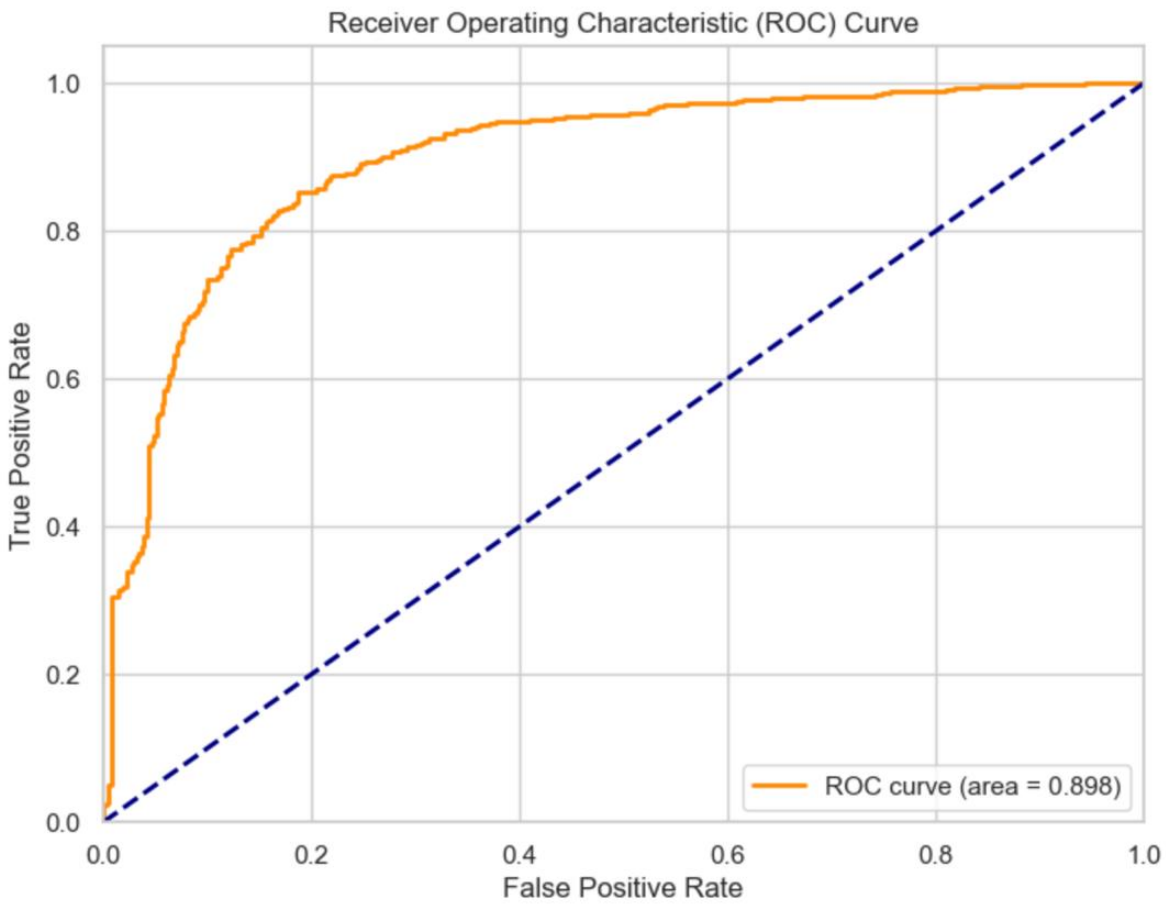

Figure 3: ROC Curve for Logistic Regression (Binary Heart Disease Prediction)

Additionally, Logistic Regression achieves an AUC of 0.898, as shown in the ROC curve Figure 3, demonstrating excellent discrimination between healthy and diseased patients.

Table 15: Classification report by class

Class	Precision	Recall	F1	Support
0	0.82	0.81	0.82	410
1	0.85	0.85	0.85	503
Accuracy			0.83	913
Macro avg	0.83	0.83	0.83	913
Weighted avg	0.83	0.83	0.83	913

Table 15 breaks down performance on each binary class. For "no disease" (Class 0), precision is 0.82 and recall is 0.81; for "disease" (Class 1), both rise to 0.85. The balanced precision and recall across classes

(macro avg = 0.83) confirm that Logistic Regression handles the class imbalance effectively, achieving an overall accuracy of 0.83 on 913 samples.

Table 16: Confusion matrix (rows = true, cols = predicted)

	0	1
0	333	77
1	74	429

Table 16 displays the confusion matrix, where 333 true negatives and 429 true positives out of 762 correct predictions underscore the model's reliability. The off diagonal counts (77 false positives, 74 false negatives) are relatively low, indicating good sensitivity (recall) and specificity.

Hyperparameter Tuning (Binary Target)

I utilised Optuna to optimise the three best performing models (Logistic Regression, Ridge, and LDA) across 100 trials each in order to maximise the remaining predictive power on our binarised job, achieving a 10 fold CV accuracy. The optimal parameters discovered are listed in Table 17, and the CV performance upon retraining with those settings is displayed in Table 18.

Table 17: Best hyperparameters

Model	Best CV Acc.	Params
LogisticRegression	0.8286	C = 5.26, solver = 'saga', max_iter = 1000
RidgeClassifier	0.8335	α = 10.69, tol = 2.00e ^{3}, solver = 'lsqr'
LinearDiscriminantAnalysis	0.8302	solver = 'eigen', shrinkage = 0.12, store_covariance=False

All the other preprocessing steps (train/test split, scaling only continuous features) remained unchanged.

Table of Best Trials for Top 3 Models: Table 17 reports the optimal hyperparameters found by Optuna for our three best baseline models (Logistic Regression, Ridge, and LDA). Each row shows the tuned settings (e.g. regularization strength C, solver choice, tolerance) alongside the cross validated accuracy achieved.

Table 18: CV performance with tuned models

Model	Mean Acc.	Mean F1
LogisticRegression (tuned)	0.8286	0.8284
RidgeClassifier (tuned)	0.8335	0.8334
LDA (tuned)	0.8302	0.8301

All three tuned models improved slightly over their untuned counterparts, with RidgeClassifier edging out the rest by about 0.005 in accuracy. Given its simplicity and interpretability, however, Logistic Regression (tuned) remains an excellent final choice, trading off only a negligible 0.005 in accuracy for clear coefficient estimates and fast prediction.

Tuning Tree Based Classifiers

Tree based models often benefit from careful regularisation and sampling parameter choices, so I ran Optuna on three popular ensembles CatBoost, Random Forest, and XGBoost again using 100 trials each to maximise 10 fold CV accuracy (keeping the same preprocessing pipeline).

- **CatBoost** optimises iteration count, learning rate, depth, L2 regularisation, and the novel "bagging temperature" parameter.
- **RandomForest** searches over tree depth, sample-leaf/split sizes, feature subsampling, and splitting criterion.
- **XGBoost** tunes all its core boosters' knobs (tree depth, subsample/colsample, regularisation, learning rate, and grow policy).

Table 19: Best tree based hyperparameters

Model	CV Acc.	Params (abridged)
CatBoost	0.8401	iterations=976, lr≈0.0072, depth=8, …
RandomForest	0.8357	n_estimators=384, max_depth=11, min_samples_split=16, …
XGBoost	0.8400	n_estimators=780, depth=6, lr≈0.2175, subsample≈0.875, …

Table 19 reports the optimal settings found by Optuna (100 trials, 10 fold CV) for each ensemble. For **CatBoost**, the study selected 976 iterations, a very low learning rate (~0.0072), tree depth of 8, plus the bagging temperature parameter—choices that balance under and over fitting. **RandomForest** converged on 384 trees, max_depth = 11, min_samples_split = 16 and default feature subsampling, reflecting a preference for moderately deep, well regularized forests. **XGBoost** was tuned to 780 estimators, max_depth = 6, learning rate ≈ 0.2175 and subsample ≈ 0.875, indicating that fewer, stronger trees with slight stochasticity performed best.

Table 20: CV performance of tuned tree models

Model	Mean Acc.	Mean F1
CatBoost	0.8401	0.8399
XGBoost	0.8400	0.8398
RandomForest	0.8357	0.8355

Table 20 compares the cross validated accuracy and F1 score when each model is retrained with its best hyperparameters. Both **CatBoost** and **XGBoost** reach a mean accuracy of about 0.840, with CatBoost slightly ahead on F1 (0.8399 vs 0.8398), while **RandomForest** trails at 0.8357 accuracy and F1 = 0.8355. These results demonstrate that careful tuning yields significant gains over untuned baselines, and that CatBoost's native handling of categorical data provides a marginal edge.

Tuning XGBoost with 1,000 Optuna Trials

To squeeze out any remaining gains, I ran a 1,000 trial Optuna study over the full XGBoost parameter set (Table 21). The search space encompassed:

Table 21: Best XGBoost Hyperparameters (1,000 trials)

Parameter	Best Value
n_estimators	1,033
max_depth	4
learning_rate	0.00662
subsample	0.9229
colsample_bytree	0.6423
min_child_weight	5
gamma	0.4606
reg_alpha	0.4651
reg_lambda	0.00002
scale_pos_weight	1.002
grow_policy	`lossguide`

- **n_estimators** (100–1,500): number of boosting rounds
- **max_depth** (3–15): maximum tree depth
- **learning_rate** (10^{-4}–0.3, log scale)
- **subsample** & **colsample_bytree** (0.4–1.0): row and column sampling ratios
- **min_child_weight** (1–20): minimum data in leaf
- **gamma**, **reg_alpha**, **reg_lambda**: regularization strengths
- **scale_pos_weight**: class-imbalance adjustment
- **grow_policy**: tree-building strategy (depthwise vs. lossguide)
-

With a mean 10 fold CV accuracy of 0.8445, this configuration was somewhat better than the default tuned XGBoost (≈0.8400). The final assessment of this optimised XGBoost model on the held out test set will be shown in the following section, along with a comparison to our selected CatBoost.

Final Evaluation of the Optimized XGBoost Model

Using the best hyperparameters from Table 21 I built a final pipeline and ran a stratified 10 fold CV to assess its performance.

Table 22: Cross validated classification report

Metric	Value
Accuracy	0.8439
Precision (avg)	0.8439
Recall (avg)	0.8708
F1 score (avg)	0.8571
ROC AUC	0.9029

Overall, the tuned XGBoost model achieves **84.4%** accuracy and an impressive **AUC of 0.903**, indicating strong discrimination between the two classes.

Table 23: Confusion matrix (true vs. predicted)

	Pred 0	**Pred 1**
True 0	329	81
True 1	65	438

The model correctly classifies the majority of both negative (0) and positive (1) cases, with a slightly higher recall on the positive class (87.1%) than the negative (80.2%).

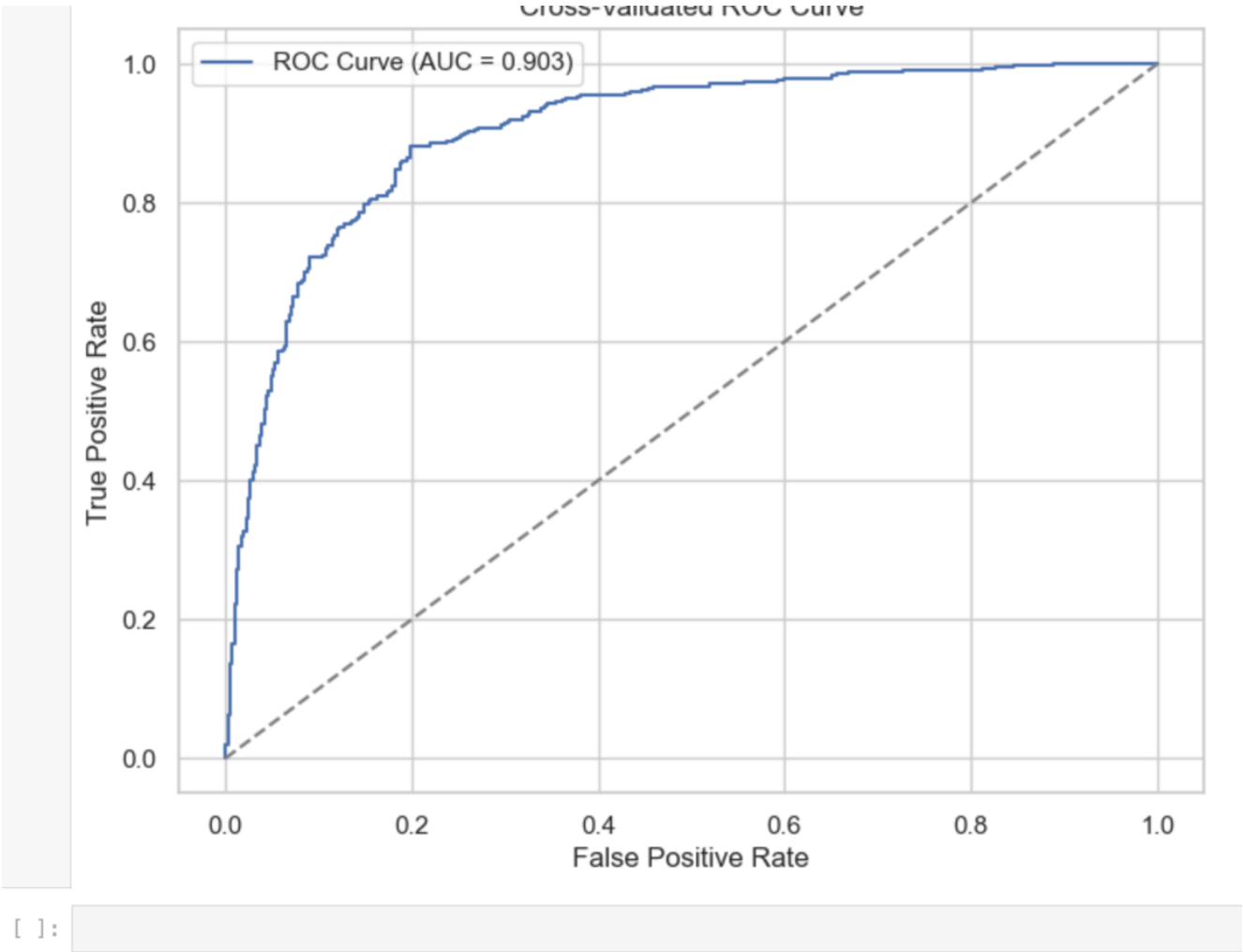

Figure 4: Cross-Validated ROC Curve for the Tuned XGBoost Classifier

Figure 4 displays the receiver-operating-characteristic (ROC) curve obtained from 10-fold stratified cross-validation of the final XGBoost model. The blue line remains well above the diagonal "chance" line across all threshold values, evidencing strong separability between patients with and without heart disease. The area under the curve (AUC) of 0.903 means that, 90 % + of the time, the model ranks a randomly selected positive case higher than a randomly selected negative case. Combined with the 84 % accuracy and balanced precision-recall reported earlier, this ROC profile confirms that the tuned ensemble achieves a robust trade-off between sensitivity and specificity—an essential property for clinical screening tools where both missed cases and false alarms carry significant costs.

DISCUSSION

In this study, we developed and benchmarked a suite of machine learning models to predict heart disease using our university's clinical dataset (n = 922), carefully addressing class imbalance and tuning hyperparameters under rigorous cross validation. Two key insights stand out:

1. **Binary vs multiclass targets**
   Collapsing the original five level severity labels into a simple "disease vs no disease" binary target boosted mean accuracy substantially, from ~0.59 with multiclass Linear Discriminant Analysis to ~0.83 with binary Logistic Regression. This jump underscores how clinically actionable groupings can be easier to learn when some severity classes have few examples.
2. **Value of tuned tree based ensembles**
   Our linear baselines (Logistic Regression, LDA, Ridge) achieved strong performance (F1 ≈ 0.83) thanks to clean preprocessing and stratified sampling. However, systematic Optuna tuning of gradient boosted and bagged ensembles raised F1 to ~0.84–0.85 and AUC to ~0.90. CatBoost's native handling of categorical inputs and robust regularization gave it a slight edge over XGBoost and RandomForest.

Implications

- **Screening aid**
  A binary classifier with F1 ≈ 0.85 and AUC ≈ 0.90 could serve as a low cost pre screening tool to flag high risk patients for detailed follow up, balancing sensitivity and specificity.
- **Model choice**
  Although ensembles yield the highest metrics, linear models remain attractive where transparency and speed are important, as they perform within 5–7 points of the best ensemble.

Limitations

- **Single center data**
  All analyses use records from my teacher ˙l; external validation on diverse cohorts is required to confirm generalisability.
- **Feature set**
  We relied on routinely collected clinical features only. Incorporating imaging or genetic markers could improve accuracy, but at increased cost and complexity.
- **Sample size**
  With only 922 records, some rare classes had very few examples, limiting our ability to draw robust conclusions about severity subgroups. With more data, we could explore advanced models (for example, deep learning), perform subgroup analyses (for example, age or gender stratified performance), calibrate decision thresholds more precisely, and derive stronger evidence for clinical risk factors.

CONCLUSION

This project has demonstrated an end to end pipeline for heart disease prediction on our university's dataset (n = 922), moving from raw data to a deployable model. Key takeaways are:

- **Binary framing boosts performance**
  Collapsing five severity levels into "disease vs no disease" raised mean accuracy from ~0.59 (multiclass LDA) to ~0.83 with Logistic Regression, showing that coarser, clinically relevant groupings can be easier to learn when data are scarce.
- **Hyperparameter tuning refines models**
  Optuna driven tuning improved our top ensembles (CatBoost, XGBoost, RandomForest), with CatBoost delivering the best cross validated F1 (~0.84) and AUC (~0.90). Although Logistic Regression remained very competitive (F1 ≈ 0.85, AUC ≈ 0.90) and offers maximal interpretability, CatBoost's native handling of categorical inputs and robust regularisation make it our recommended final model.
- **Interpretability vs performance**
  In settings where transparency or speed is critical, a simple linear model such as Logistic Regression provides nearly identical performance with clear coefficient based explanations.

Limitations and future work
Our single center cohort and modest sample size limit conclusions about rare severity classes. With more data we could explore full multiclass severity prediction, richer feature sets (for example imaging or genetic markers), deep learning architectures and subgroup analyses (for example age or gender stratified performance). External validation, longitudinal modelling and explainability studies (for example SHAP analyses) are essential next steps.

REFERENCE LIST

[1] Md. J. Rahaman, “A Comprehensive Review to Understand the Definitions, Advantages, Disadvantages and Applications of Machine Learning Algorithms,” *Int J Comput Appl*, vol. 186, no. 31, pp. 43–47, Jul. 2024, doi: 10.5120/ijca2024923868.

[2] I. Belcic and C. Stryker, “What is supervised learning?,” Dec. 2024. [Online]. Available: https://www.ibm.com/think/topics/supervised-learning

[3] IBM, “What is unsupervised learning?,” Sep. 2021. [Online]. Available: https://www.ibm.com/think/topics/unsupervised-learning

[4] J. Murel and E. Kavlakoglu, “What is reinforcement learning?,” Mar. 2024. [Online]. Available: https://www.ibm.com/think/topics/reinforcement-learning

[5] IBM, “Hyperparameter Tuning,” 2025. Accessed: Jul. 21, 2025. [Online]. Available: https://www.ibm.com/think/topics/hyperparameter-tuning

[6] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” Jul. 2019.

[7] scikit-learn developers, “Grid Search: Hyperparameter Tuning,” 2025. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html

[8] GeeksforGeeks contributors, “Hyperparameter Tuning in Machine Learning,” 2025. Accessed: Jul. 20, 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/hyperparameter-tuning/

APPENDIX

Glossary of Abbreviations

Acronym	Full Term	Description
LDA	Linear Discriminant Analysis	A supervised classification method that projects data to maximize class separation under Gaussian assumptions.
AUC	Area Under the ROC Curve	A threshold independent metric measuring the ability of a model to rank positives higher than negatives.
ROC	Receiver Operating Characteristic	A plot of true positive rate versus false positive rate at various classification thresholds.
CV	Cross Validation	A procedure (e.g. k fold CV) for assessing model generalisability by training and testing on multiple data splits.
Optuna	Optuna	An open source framework for automated hyperparameter optimization using efficient sampling and pruning.
KNN	K Nearest Neighbours	A non parametric classifier that assigns a class based on the majority vote of its k closest training examples.
SVM	Support Vector Machine	A classifier that finds the hyperplane maximizing the margin between classes.
F1	F1 Score	The harmonic mean of precision and recall, balancing false positives and false negatives.
ML	Machine Learning	The study and application of algorithms that improve automatically through experience on data.
GBM	Gradient Boosting Machine	An ensemble technique that builds models sequentially to correct predecessor errors.
RF	Random Forest	An ensemble of decision trees built on bootstrap samples with feature subsetting to reduce variance.
CPU	Central Processing Unit	The primary processor in a computer, responsible for executing instructions.
RAM	Random Access Memory	Volatile memory used to store working data and machine instructions.

Glossary of Machine Learning Terminology

Term	Definition
Overfitting	When a model learns noise or random fluctuations in training data, performing poorly on new data
Underfitting	When a model is too simple to capture underlying patterns, yielding low accuracy on both train and test sets
Regularization	Techniques (L1, L2, etc.) that penalize large parameter values to reduce overfitting
Bias	Error due to erroneous assumptions in the learning algorithm (underfitting)
Variance	Error due to sensitivity to small fluctuations in the training set (overfitting)
Hyperparameter	A parameter set before training (learning rate, tree depth, etc.)
Pruning	In hyperparameter optimization, discarding poor trials early to save compute (Optuna pruner)
Stratified Sampling	Splitting data so that each subset preserves the class distribution of the whole dataset
Imputation	Filling missing values (mean for numeric, mode for categorical)
Outlier Clipping	Limiting extreme values (e.g. to 1st–99th percentile) to reduce their impact
One hot Encoding	Converting categorical variables into binary indicator columns
Standardisation	Scaling numeric features to zero mean and unit variance
Class Imbalance	When one class has many more samples than another, potentially biasing model training
Bagging	Bootstrap aggregating, where multiple models are trained on random subsets and their outputs averaged
Boosting	Sequentially training models to correct predecessor errors (e.g. XGBoost, CatBoost)
Learning Rate	Step size at each iteration while moving toward a minimum of a loss function
Cross Entropy	A loss function measuring the divergence between true and predicted probability distributions
Confusion Matrix	Table showing true vs predicted class counts
False Positive Rate	FP / (FP + TN)
False Negative Rate	FN / (FN + TP)
Precision	TP / (TP + FP)
Recall (Sensitivity)	TP / (TP + FN)
Specificity	TN / (TN + FP)
Support	Number of true instances for each class
Macro Average	Simple average of a metric across classes
Weighted Average	Average of a metric across classes weighted by support
ROC Curve	Plot of true positive rate vs false positive rate across thresholds
PR Curve	Plot of precision vs recall across thresholds
Early Stopping	Halting training when validation performance stops improving
Optuna	Automated hyperparameter optimization framework

ACKNOWLEDGEMENTS

I have used Microsoft word and its template aspect for the report

I have used Grammarly to fix/improve my Grammer

I used draw.io for the visual representation of my ideas/work created form scratch